

Statistics cookbook — Econometrics & Finance

For: economics, applied econometrics, quantitative finance, and business analytics — fields whose work is built on **time series, returns, and observational regression**, and whose central worries are *non-stationarity*, *serial correlation*, and *heteroskedasticity* rather than experimental control.

This is a **task-first** guide: find the question you're asking, copy the recipe, read how to report it. Every snippet here is executed by `examples/cookbook_econ_test.bas` (run by the test suite), so the code is known-good, and every statistic is verified to match SciPy / statsmodels exactly.

Setup

```
program analysis(args)
  load stats from "stdlib/stats.bas"      ' path relative to your program
  ' ... recipes below ...
end program
```

Run with the stdlib on the path: `GBASIC_PATH=stdlib ./gbasic analysis.bas` (after `make install`, `load stats from "stats.bas"` resolves on its own).

Data conventions

- A **series** (prices, returns, a variable) is a flat list, in time order: `[100.0, 102.0, 101.0, ...]`.
- Predictors** for a regression are a list of columns: `[x1, x2]` (each a list). A single predictor is `[x]`.
- Any function returns a **record** of results, or `unknown` on bad/degenerate input — guard with `is_unknown(...)`.

1. Returns from prices

```
r = simple_returns(prices)      ' r_t = P_t/P_{t-1} - 1    (length n-1)
lr = log_returns(prices)       ' r_t = ln(P_t/P_{t-1})
tot = cumulative_return(r)     ' compounded: prod(1+r) - 1
```

Use **log returns** for statistical modeling (they add across time); **simple returns** for reporting performance.

2. Performance & risk metrics

```
sharpe_ratio(r, rf, periods)    ' annualized; periods = 252 daily, 12 monthly
sortino_ratio(r, rf, periods)   ' downside-only volatility in the denominator
md = max_drawdown(prices)      ' {max_drawdown, peak, trough}; drawdown is negative
value_at_risk(r, 0.05, "historical") ' 95% VaR, as a positive loss; or "normal"
cvar(r, 0.05)                  ' expected shortfall (average tail loss)
```

Report: annualized Sharpe (`rf` is the per-period risk-free rate), max drawdown as a percentage, and VaR/CVaR as the loss you'd expect to breach `alpha` of the time. Sharpe uses total volatility; Sortino penalizes only downside

moves.

3. Systematic risk (CAPM / market model)

Question: how much of an asset's excess return is explained by the market?

```
cp = capm(asset_returns, market_returns, rf)
cp.beta      ' systematic risk (market sensitivity)
cp.alpha     ' Jensen's alpha (skill / mispricing); cp.alpha_p to test it vs 0
cp.r_squared  ' fraction of variance explained by the market
```

Report: $\beta = \text{cp.beta}$ (SE cp.beta_se), $\alpha = \text{cp.alpha}$ with its p -value — a significant positive α is the headline claim of "outperformance." Extend to multi-factor (Fama-French) models with `ols(asset_excess, [mkt, smb, hml])`.

4. Regression with robust inference

Ordinary standard errors assume homoskedastic, uncorrelated errors — rarely true for economic data. Match the standard error to the violation:

```
fit = ols(y, [x1, x2])          ' point estimates (always valid under OLS)
rob = ols_robust(y, [x1, x2], "HC3") ' heteroskedasticity-robust SEs
nw = newey_west(y, [x1, x2], 4)  ' HAC: robust to heteroskedasticity +
autocorrelation
```

- Cross-section, unequal variance → `ols_robust` (HC0/HC1/HC2/HC3; HC3 is the safe default in small samples).
- Time series with serially correlated errors → `newey_west` with `maxlags` (a common rule is $\lfloor 4 \cdot (n/100)^{(2/9)} \rfloor$).

The **coefficients are identical** across all three; only the SEs / p -values change. **Report** the coefficient with the robust SE and say which estimator you used and why.

5. Diagnosing the regression

```
breusch_pagan(fit.residuals, [x1, x2]) ' heteroskedasticity: small p -> non-constant
variance
ljung_box(fit.residuals, 4)             ' leftover autocorrelation: small p -> correlated
errors
durbin_watson(fit.residuals)            ' ~2 = no AR(1); <2 positive, >2 negative
```

Report: if Breusch-Pagan rejects, use `ols_robust` / `newey_west`; if Ljung-Box or Durbin-Watson flags autocorrelation, use `newey_west` (or model the dynamics — §7). Each returns `{statistic, df, p_value}` (Durbin-Watson returns a bare statistic).

6. Is the series stationary? (unit-root testing)

Do this first for any time-series model — regressing one non-stationary series on another produces *spurious* results.

```
a = adf_test(x, lags, "c")      ' Augmented Dickey-Fuller; trend "n"/"c"/"ct"
a.statistic    a.p_value    a.crit_5      ' reject unit root (stationary) if p small / stat
< crit
```

If it fails to reject (large p), **difference and retest**:

```
dx = diff(x, 1)                ' first difference
a2 = adf_test(dx, lags, "c")    ' usually stationary now -> the series is I(1)
```

Report: the ADF statistic, its p -value, and the differencing order d you settled on. `trend="c"` for a series around a constant mean, `"ct"` if it drifts with a linear trend, `"n"` for zero-mean.

7. Modeling a time series (ARIMA family)

Identify the order from the autocorrelation functions, then fit:

```
acf(x, 20)      ' MA signature: cuts off after lag q
pacf(x, 20)     ' AR signature: cuts off after lag p

m = arima_fit(x, p, d, q)      ' d = differencing from §6
m.phi    m.theta    m.aic    m.bic
fc = arima_forecast(m, x, h)   ' h-step-ahead forecast
```

Special cases: `ar_fit(x, p)` (pure AR by OLS), `arma_fit(x, p, q)` (exact Kalman-filter MLE, matches statsmodels). **Report:** the chosen (p, d, q) , the coefficients, AIC/BIC for model comparison, and — crucially — confirm the residuals are white noise with `ljung_box` (§5). Compare candidate orders by **lowest AIC/BIC**.

8. Volatility clustering (ARCH / GARCH)

Financial returns have calm and turbulent periods — variance that clusters in time. Test for it, then model it:

```
arch_lm(returns, 2)            ' Engle's test; small p -> ARCH effects present
g = garch_fit(returns)          ' GARCH(1,1)
g.omega    g.alpha    g.beta    g.persistence    ' persistence = alpha+beta (near 1 = long
memory)
```

Report: the ARCH-LM test result (justifies modeling volatility at all), then the GARCH parameters. `alpha` is the reaction to shocks, `beta` the persistence of past variance; `alpha + beta` close to 1 means volatility shocks decay slowly. (GARCH is verified against the `arch` package.)

9. A typical workflow

1. **Prices** → **returns** (`log_returns`), describe with §2 metrics.
2. **Stationarity** (`adf_test`); difference until stationary (§6).
3. **Identify** order from `acf` / `pacf` (§7).
4. **Fit** `arima_fit` ; compare orders by AIC/BIC.
5. **Check residuals** are white noise (`ljung_box` , `durbin_watson` , §5) and test for ARCH (`arch_lm` , §8).
6. If volatility clusters, **add** `garch_fit` ; if you only need robust regression inference, use `newey_west` (§4).

10. Reporting quick reference

You have	Recipe	Headline numbers
A price series	<code>simple_returns</code> / <code>log_returns</code>	returns, cumulative return
Return series	<code>sharpe_ratio</code> / <code>sortino_ratio</code>	annualized ratio
Downside risk	<code>value_at_risk</code> / <code>cvar</code>	VaR, expected shortfall
Drawdown	<code>max_drawdown</code>	worst peak-to-trough %
Asset vs market	<code>capm</code>	β , $\alpha(p)$, R^2
Regression, unequal variance	<code>ols_robust</code>	b , robust SE, p
Regression, serial correlation	<code>newey_west</code>	b , HAC SE, p
Heteroskedasticity check	<code>breusch_pagan</code>	LM, p
Autocorrelation check	<code>ljung_box</code> / <code>durbin_watson</code>	Q/p , DW
Unit root / stationarity	<code>adf_test</code>	ADF stat, p , crit
Series dynamics	<code>arima_fit</code> + <code>arima_forecast</code>	(p,d,q), AIC, forecast
Volatility clustering	<code>arch_lm</code> + <code>garch_fit</code>	ARCH p , $\alpha+\beta$

All results are verified numerically against SciPy / statsmodels (GARCH against the `arch` package). See [docs/statistics_scientist_plan.md](#) for method notes and [examples/cookbook_econ_test.bas](#) for the runnable version of every recipe above. Predictive machine learning (train/test, cross-validation, classifiers) is a separate track, intentionally out of scope here.